

## 1) Declara las variables de estado

Crea los arreglos y variables que controlarán la secuencia del juego y el estado actual.

```
const buttonColors = ["red", "blue", "green", "yellow"];
let gamePattern = [];           // secuencia que genera el juego
let userClickedPattern = [];    // secuencia que va el usuario
let started = false;           // si el juego inició
let level = 0;                  // nivel actual
```

## 2) Inicia el juego al presionar una tecla (o botón)

Agrega un listener para empezar la partida (solo la primera vez).

```
document.addEventListener("keydown", () => {
  if (!started) {
    level = 0;
    gamePattern = [];
    started = true;
    nextSequence(); // empieza la primera secuencia
    document.getElementById("level-title").textContent = "Nivel " +
level;
  }
});
```

## 3) Detecta los clics del usuario en las piezas (botones)

Asigna manejadores a cada botón para guardar la elección y reaccionar visual/sonoramente.

```
document.querySelectorAll(".btn").forEach(btn => {
  btn.addEventListener("click", function() {
    const userChosenColor = this.id;           // id debe ser
'red','blue','green','yellow'
    userClickedPattern.push(userChosenColor);
    playSound(userChosenColor);
    animatePress(userChosenColor);
    checkAnswer(userClickedPattern.length - 1);
  });
});
```

## 4) Implementa nextSequence() — generar el siguiente color

Genera un color aleatorio, lo agrega a la secuencia y muestra la animación para el usuario.

```
function nextSequence() {
  userClickedPattern = [];      // reinicia la secuencia del usuario
  level++;
  document.getElementById("level-title").textContent = "Nivel " +
level;

  const randomNumber = Math.floor(Math.random() * 4);
  const randomChosenColor = buttonColors[randomNumber];
  gamePattern.push(randomChosenColor);

  // muestra la animación y sonido del color añadido
  const btn = document.getElementById(randomChosenColor);
  btn.classList.add("pressed");
  playSound(randomChosenColor);
  setTimeout(() => btn.classList.remove("pressed"), 300);
}
```

## 5) Implementa checkAnswer(currentIndex) — comparar elección del usuario

Verifica si la última pulsación coincide con la secuencia del juego.

```
function checkAnswer(currentIndex) {
  if (gamePattern[currentIndex] ===
userClickedPattern[currentIndex]) {
    // si el usuario completó la secuencia correctamente
    if (userClickedPattern.length === gamePattern.length) {
      setTimeout(() => nextSequence(), 1000);
    }
  } else {
    // error: reproducir sonido y mostrar efecto de 'game over'
    playSound("wrong");
    document.body.classList.add("game-over");
    document.getElementById("level-title").textContent = "¡Perdiste!
Presiona una tecla para reiniciar";
  }
}
```

## Simon Dice Game

```
        setTimeout(() => document.body.classList.remove("game-over"),
200);
        startOver();
    }
}
```

## 6) Implementa startOver() — reiniciar variables

Reinicia el estado del juego para una nueva partida.

```
function startOver() {
    level = 0;
    gamePattern = [];
    started = false;
    userClickedPattern = [];
}
```

## 7) Función playSound(name) — reproducir sonidos

Usa Audio para reproducir sonidos según el color o sonido de error.

```
function playSound(name) {
    const sounds = {
        red: new
Audio("https://s3.amazonaws.com/freecodecamp/simonSound1.mp3"),
        blue: new
Audio("https://s3.amazonaws.com/freecodecamp/simonSound2.mp3"),
        green: new
Audio("https://s3.amazonaws.com/freecodecamp/simonSound3.mp3"),
        yellow: new
Audio("https://s3.amazonaws.com/freecodecamp/simonSound4.mp3"),
        wrong: new
Audio("https://s3.amazonaws.com/adam-recv1ohe-sounds/error.mp3")
    };
    sounds[name].play();
}
```

## 8) Función `animatePress(color)` — efecto visual al presionar

Añade y quita una clase para el efecto visual.

```
function animatePress(color) {  
  const btn = document.getElementById(color);  
  btn.classList.add("pressed");  
  setTimeout(() => btn.classList.remove("pressed"), 200);  
}
```